

Slot-Chain: A Preconfirmation Protocol

Production Design Candidate — v2.3

August 2026

Abstract

Slot-Chain assigns one-second L2 slots to bonded builders. Builders sign parent-linked blocks off chain; any party can prove and submit a batch to L1. Normal candidates compete for a fixed settlement interval. After an objective finality breach, or when a forced L1 message becomes due, anyone can prove an unsigned deterministic escape block. Recovery is an episode of renewable, objectively expiring rounds, so a prover outage cannot make its target stale forever. This revision separates proved outputs from L1 close metadata, replaces the unauthenticatable queue hash chain with a range-provable Merkle vector, makes forced-message execution Ethereum-valid, and specifies bounded admission, data, reward and migration state. It is an implementation candidate, not a production authorization: the proving, gas, economics and adversarial gates in §13 remain mandatory.

Contents

1	Status, claims and exclusions	3
1.1	Claims	3
1.2	Non-claims	3
2	Actors and trust model	3
3	Clock domains and canonical state	4
4	Builder registry and exact lookahead	5
4.1	Registry lifecycle	5
4.2	Authenticated snapshot	5
4.3	Consensus byte encoding and allocation	6
5	Blocks, promises and proof statement	7
5.1	Signed header	7
5.2	Three validity tiers	8
5.3	Verifier statement ABI	8
5.4	Preconfirmation semantics	9
6	Blob data authentication	10
7	Settlement and recovery state machine	11

7.1	Normal candidates	11
7.2	Activation	12
7.3	Recovery acceptance	12
8	Forced queue and unsigned escape	13
9	Economics, slashing and payments	15
10	Security and liveness analysis	15
11	Implementation blueprint	16
11.1	L1 modules and bounded storage	16
11.2	Historical authentication	17
11.3	Reorg rule	17
11.4	Genesis and migration	17
12	Parameters and enforced geometry	18
13	Production gates and verdict	19
A	Normative commitment encodings	21
B	Normative transition ordering	23
C	Rejected alternatives	24
D	Executable consensus models	25
E	Security invariants checklist	25

1 Status, claims and exclusions

Normative words **MUST**, **MUST NOT** and **SHOULD** have their usual RFC meaning. Where prose and either executable model disagree, deployment is blocked until the disagreement is resolved; neither source silently overrides the other.

1.1 Claims

Subject to L1 safety/liveness and a sound validity-proof system, the protocol claims:

1. **Finalized-state safety.** L1 accepts only a proved state transition extending the stored canonical tuple. Builder signatures never substitute for execution validity.
2. **Bounded protocol recovery.** A builder cartel, an absent aggregator, an empty builder registry, and missing standbys cannot prevent an unsigned escape candidate from advancing the canonical state.
3. **Forced transaction inclusion.** A correctly prepaid L1 envelope at the queue head is consumed by the next canonical block after it becomes due; it is executed if valid and consumed-and-discarded if invalid.
4. **Permissionless landing and recovery.** No allowlist, builder key, aggregator seat or payment recipient is required to submit a proof or an escape candidate.
5. **Deterministic lookahead.** Every implementation derives the same schedule from an authenticated finalized snapshot and the byte encoding in §4.

1.2 Non-claims

- A preconfirmation is not finality or insurance. Before normal close it may be revoked by builder equivocation, an unprovable skip, a withheld body, or an escape commit. Applications must price this as probabilistic/economic assurance and publish their own exposure limit.
- A bond cannot cap aggregate off-chain promises: the protocol has no reservation ledger and a builder can sign mutually exclusive promises. L_LEASE is deterrence and a recovery source, not guaranteed coverage.
- Protocol liveness is a capability claim. Economic liveness additionally assumes at least one actor can fund proof generation and an L1 transaction. A payment failure never invalidates an otherwise valid canonical transition.
- Censorship by L1 for longer than T_INCLUDE_MAX_SECONDS, failure of L1 finality, or a broken proof system is outside the model.

2 Actors and trust model

Builder.

A registered address eligible for scheduled slots. Builders may equivocate, skip parents, withhold bodies, collude, or disappear.

Aggregator.

The current auction seat, expected to collect data, prove and submit normal candidates. It has no exclusive consensus authority.

Prover/lander.

Any address. It supplies a validity proof and names an optional reward beneficiary in the proof statement. Proof validity, not identity, authorizes transition.

Forced-message sender.

Any L1 account that submits a gas-bounded, prepaid L2 transaction envelope.

L1.

Ethereum provides canonical ordering, timestamps, EIP-4788 beacon roots, EIP-2935 historical block hashes, blob commitments and the KZG point-evaluation precompile.

The safety threshold contains no honest-builder assumption. Honest builders improve normal throughput and preconfirmation reliability; the unsigned escape lane provides state and forced-queue progress when all of them collude. The L1-liveness assumption includes transaction inclusion and at least 64 canonical L1 blocks becoming available within $T_DEPTH_MAX=900s$; a longer L1 stall suspends, rather than violates, the L2 recovery clock.

3 Clock domains and canonical state

Before activation, Settlement is in `PREACTIVE`; candidate submission is disabled and slot arithmetic saturates. L2 slot zero is fixed by `GENESIS_TIMESTAMP`, with

$$\text{currentL2Slot} = \max(0, \text{block.timestamp} - \text{GENESIS_TIMESTAMP}).$$

L2 slot and every deadline are timestamp seconds. L1 confirmation depth is counted in L1 block numbers. Beacon slot is used only in schedule authentication. Implementations **MUST NOT** substitute one clock for another.

The L1 contract stores a proved core and local L1 metadata:

```
CanonicalCore = (tipHash, tipSlot, stateRoot, messageCursor,
                 winningDataCommitment)
Canonical = (CanonicalCore core, uint64 canonicalizedAtBlock)
```

The proof binds `baseCanonicalHash`, the hash of both stored objects, but outputs only a new `CanonicalCore`. Settlement compares every base field with storage, verifies the proof, writes the explicit proved core, and only then sets `canonicalizedAtBlock=block.number`. A prover is therefore never asked to predict the L1 block in which its transaction will land. No code attempts to read or store `blockhash(block.number)`, which does not exist during that block.

A candidate has one prior-block anchor. Normal anchors are authenticated by an EIP-2935 block hash, the supplied execution header and MPT proofs, including the historical forced-queue root/count. A recovery round instead stores `blockhash(block.number-1)` and its timestamp while available, directly stores the current queue root/count, and requires the parent anchor to reach depth `F_L1`. Current queue state is never claimed to be part of the parent's historical state root. An L1 reorg rolls the stored canonical state and anchor back together. Initial values and bridge cursors are migration inputs in §11.

Every state-mutating external entry point begins with `sync()`. If `sync()` changes mode or commits a mature normal candidate, the wrapper **MUST** return `SYNCED` successfully before parsing or validating the caller's candidate. The caller resubmits against the new version. This rule is necessary because a later Solidity revert would otherwise roll back the `sync` transition in the same transaction.

4 Builder registry and exact lookahead

4.1 Registry lifecycle

A registration contains a nonzero address, a uint192 bond not exceeding BOND_MAX, a monotonic uint64registrationIndex, an effectiveL2Slot, and separately reserved L_LEASE[window] tranches. Address zero is permanently VACANT. Duplicate live addresses and re-registration of a tombstoned key are forbidden.

The active table has exactly 64 indexed cells. A registration becomes effective only after ENTRY_DELAY_WINDOWS=8. If the table is full it may replace only the smallest-bond effective live entry, breaking ties by greatest registration index, and only with a strictly greater bond. At most MAX_GENERATION_MOVES_PER_WINDOW=4 replacements or effective exits are processed. A displaced generation is moved, with its tranche root and bond, into the fixed MAX_LIABILITY_GENERATIONS=1,072 liability ring; an occupied ring cell is never overwritten before its release predicate holds. Registration therefore rejects, rather than losing slashable history, when the replacement budget or ring is full.

Every admission-affecting mutation increments admissionVersion and updates two roots: a six-level registryRoot over the 64 active cells and an eleven-level admissionRoot over 2,048 fixed positions containing the 64 active and 1,072 liability records plus canonical empty padding. The latter retains displaced scheduled keys. Normal open and every recovery round freeze the pair (admissionVersion, admissionRoot). Each scheduled signer carries an inclusion proof for a non-tombstoned record effective at that L2 slot; a slash cannot retroactively invalidate an already frozen window.

Eligibility is evaluated before ranking. A cell's effective L2 slot must be no later than the source-derived L2 slot, its fully reserved tranche for W must be proven, and its tombstone-effective L2 slot must be later than the source. All 64 active cells, including empty or FREE tranche leaves, are supplied. The 64 eligible entries with greatest bond, then smallest registration index, form the snapshot set. Supplying proofs only for purported winners is invalid because it cannot prove omission-free ranking.

An exit request becomes eligible for processing only after EXIT_DELAY_WINDOWS=MAX_LIVE_WINDOWS; it may wait behind the same bounded generation-movement queue, and the bond remains escrowed until every generation and tranche is releasable. A tranche follows FREE->RESERVED(W)->LIABLE(W)->RELEASED or RESERVED/LIABLE->SLASHED. Every candidate block slot must be at least the normal window's frozen minimum (or the recovery round start), not merely its tip. Let windowEndSlot(W)=384*(W+1)-1. Release requires windowEndSlot(W)<globalMinReferencedSlot, no stored best or active round references it, and the absolute evidence and L1-reorg deadlines have expired. At that point maintenance atomically marks the schedule cell EXPIRED and may release its tranches; an unexpired sealed record is never overwritten. globalMinReferencedSlot is the minimum of the current dynamic floor, an open normal window's frozen floor, an active round's start and the first slot of any stored best (absent values are ignored). Slot indices are never compared with timestamps or window numbers.

4.2 Authenticated snapshot

For aligned window W, let its start timestamp be GENESIS_TIMESTAMP+384W. The target is the greatest BEACON_GENESIS_TIME+12k not later than eight L1 epochs before that start; L2-genesis alignment is irrelevant. Define sealDeadline(W)=windowStart-H_LOOK. A seal is valid only when its carrier execution block has F_FINAL L1-block depth and

`block.timestamp < sealDeadline(W)`. Beacon slots are not used as a surrogate for execution-block depth. At or after the deadline an unsealed window is permanently all-VACANT; a late transaction cannot change it. The contract performs this procedure:

1. The contract itself, not caller data, queries EIP-4788 for $j = 1, \dots, 64$ at $q = \text{targetTimestamp} + 12j$; missing timestamps revert and are skipped. The first successful query identifies the *carrier* execution block. EIP-4788 returns its parent beacon-block root, not the carrier root.
2. For the first successful query the caller supplies the carrier execution header. EIP-2935 authenticates its hash and block number; the header timestamp equals q and its `parent_beacon_block_root` equals the EIP-4788 result. Fork-specific SSZ branches in that parent authenticate its slot and one execution payload's `blockHash`, `stateRoot`, `prevRandao` and timestamp. The parent slot must be at or before target, its payload timestamp must equal its beacon-slot time, and its payload block hash must equal the carrier header's parent hash. No carrier observed by all 64 contract calls means all VACANT. If any call succeeds, a malformed header, a later parent, or a fork/gindex mismatch reverts without recording state; adversarial witness bytes can never manufacture a vacant seal.
3. Verify Merkle–Patricia storage proofs from that execution state root for the registry header and `registryRoot`. The caller supplies all 64 canonical serialized cells, including empty cells; recompute their six-level Keccak tree and require equality to `registryRoot`. Filter eligibility and order the entries, then store the byte-exact 64-leaf `entryRoot` and seed. Every active cell supplies its tranche leaf at index $W \bmod 512$, including the canonical empty/FREE leaf, and the leaf's stored window must be W . The fixed leaf count proves completeness with one MPT path rather than 64 independent storage paths. The carrier satisfies `currentExecutionBlock-carrierBlockNumber >= F_FINAL`; the source payload is necessarily older.

`sealWindow` is permissionless. An optional maintenance distributor may pay its event in a later transaction. A missing seal fails closed to VACANT; it cannot give an address an unauthorized ticket. The snapshot age plus 64-slot carrier scan, finality and seal margin fits the eight-epoch geometry and remains far below EIP-4788's 8,191-slot history window. Consensus constants include the Deneb SSZ generalized indices; an L1 fork that changes them requires a protocol-version upgrade, never runtime guessing.

For Deneb, Electra and Fulu, the generalized index from `BeaconBlock` root to slot is 8 and to `body.execution_payload` is 201. Within the composed tree, payload `state_root`, `prev_randao`, `timestamp` and `block_hash` are 6,434, 6,437, 6,441 and 6,444 respectively. The seal input names the L1 fork digest; only these configured digests are accepted. Gloas or any fork that changes the container requires new constants, fixtures and a protocol-version upgrade before its activation.

4.3 Consensus byte encoding and allocation

All hashes use Ethereum legacy Keccak-256. Concatenation below is packed fixed width; strings are the literal ASCII bytes without a length prefix:

$$\begin{aligned}
seed_W &= H(\text{"slot-chain-seed-v1"} \parallel u256(chainId) \\
&\quad \parallel u256(protocolVersion) \parallel u256(W) \parallel bytes32(prevRandao)), \\
qTie_i &= H(\text{"slot-chain-quota-tie-v1"} \parallel seed_W \parallel u64(regIndex_i)), \\
sTie_{p,i} &= H(\text{"slot-chain-slot-order-v1"} \parallel seed_W \parallel u16(p) \parallel address20(i)).
\end{aligned}$$

Starting from zero, capped D'Hondt awards each of 384 tickets to the unsaturated entry maximizing $\text{bond}/(\text{allocation}+1)$, with $Q_MAX=76$. Quotients are compared by cross multiplication; a uint192 bond times Q_MAX+1 cannot overflow uint256 . Equal quotients use smallest $qTie$. Remaining tickets are VACANT; six addresses are required to fill a window.

Placement processes positions in order. A real address is feasible if it has a ticket remaining, differs from the preceding real address, and removing it preserves $\text{remaining}[a] \leq \text{otherRemaining}+1$ for every real address. Choose the feasible value with smallest $sTie$; VACANT may repeat. The executable model and its golden vectors are normative test fixtures.

5 Blocks, promises and proof statement

5.1 Signed header

A builder signs exactly:

```
(chainId, protocolVersion, verifyingContract, slot, parentHash,
blockHash, stateRoot, bodyRoot, anchorNumber, anchorHash,
forceRoot, forceCutoff, messageStart, messageEnd, dataManifestRoot,
coinbase, tier, admissionVersion, episode, recoveryRevision, recoveryId)
```

The normative EIP-712 type string is:

```
SlotChainBlock(
uint256 chainId,uint256 protocolVersion,address verifyingContract,
uint64 slot,bytes32 parentHash,bytes32 blockHash,bytes32 stateRoot,
bytes32 bodyRoot,uint64 anchorNumber,bytes32 anchorHash,bytes32 forceRoot,
uint64 forceCutoff,uint64 messageStart,uint64 messageEnd,
bytes32 dataManifestRoot,address coinbase,uint8 tier,
uint64 admissionVersion,uint64 episode,uint64 recoveryRevision,
bytes32 recoveryId)
```

The hashed ASCII has one space between each field type and field name and no other whitespace; displayed line breaks, indentation and spaces after commas are typography only. The exact single-line literal and a standalone Keccak implementation are in `commitment-model.py`. TYPEHASH is legacy Keccak-256 of those ASCII bytes and equals the concatenation of these two fragments:

```
0x0a0bfc3b8ba52a166f662055d0989146
5d916aa486808e78105890e98d65777f
```

The EIP-712 domain is `("SlotChain", "2", chainId, verifyingContract)`. Signatures are exactly 65-byte `r||s||v`; `v` is 27 or 28, `s` is low, and zero or recovered-zero addresses reject.

EIP-2098 and EIP-155 transaction-style `v` values are not accepted. Tier 1 requires `episode=recoveryRevision=0` and `recoveryId=0`; tier 2 requires the exact live round values. Consequently semantically unused bytes cannot create false equivocation evidence. The execution `blockHash` excludes the off-chain builder signature, avoiding a circular hash definition. The signed `bodyRoot` covers only canonical discretionary transaction bytes; the proof derives the full Ethereum transactions root after adding the profile-defined system and valid forced transactions.

5.2 Three validity tiers

1. **Normal signed.** Every block is signed by its scheduled builder under the normal window's frozen (`admissionVersion`, `admissionRoot`). Slots strictly increase, each slot is no later than `currentL2Slot+CLOCK_SKEW`, and the first slot strictly exceeds the canonical base slot. Every block is at or above the window's frozen `minAdmissibleSlot`; parent gaps are at most `G_MAX=64`.
2. **Recovery signed.** The first block extends the L1-final canonical tuple, binds the active episode and current recovery revision, and may cross an unbounded time gap. Every block still requires its scheduled builder signature under the current recovery round's frozen admission pair. Every slot is at or after that round's `roundStartSlot`.
3. **Escape unsigned.** Exactly one deterministic block extends the L1-final canonical tuple during recovery. It has no builder signature, no beneficiary-controlled coinbase, no discretionary transaction and no blob body. It contains only the protocol anchor and the deterministic maximum prefix of the current round's frozen forced queue. When the queue is empty an SLA-triggered episode may commit an empty anchor-only escape block. Its slot is `max(roundStartSlot+ESCAPE_OFFSET, canonical.tipSlot+1)` and its anchor, queue root/cutoff, block hash and state root are unique functions of the current recovery round and proved execution result.

Every candidate uses exactly one `batchAnchor`; every block repeats that anchor and frozen `forceRoot/forceCutoff`. For a normal candidate the supplied RLP execution header hashes to the EIP-2935 value, its timestamp is no later than the first L2 block timestamp, and MPT proofs under its state root authenticate the historical forced queue and any other contract state. For recovery, the RLP header only authenticates the stored parent hash/timestamp; the queue pair is authenticated directly by the round fields and is not asserted against that header's state root. Requiring one anchor, not up to 4,096 independent anchors, bounds both circuit and L1 work. Cutoffs therefore do not change within a candidate; the first blocks drain the frozen FIFO prefix and later blocks see an empty prefix.

The circuit verifies parent linkage, strict slot monotonicity, schedules, signatures for tiers 1–2, version binding, anchor/header/MPT authentication, forced-prefix ordering, execution, exact body/data coverage and public outputs. It rejects more than 4,096 blocks, 12 schedule windows, 16 sealed sessions, 2,100 referenced blob records, 256 forced items, 4 MiB forced bytes or 80 million total accounted forced gas per candidate. These candidate-level caps are independent of the per-block caps.

5.3 Verifier statement ABI

The L1 verifier accepts `verify(proof, uint256[2] digestLimbs)`. Settlement constructs the following Solidity struct in exactly this order and computes `statementHash` as legacy Keccak

of the ASCII domain "slot-chain-statement-v1" followed by `abi.encode(S)`:

```
S = (uint256 chainId, uint256 protocolVersion,
    address verifyingContract,
    uint8 tier, bytes32 baseCanonicalHash,
    bytes32 candidateCommitment, uint64 blockCount,
    uint64 firstSlot, bytes32 tipHash, uint64 tipSlot,
    bytes32 endStateRoot, uint64 endCursor,
    bytes32 winningDataCommitment, uint64 nextDueAt,
    uint64 anchorNumber, bytes32 anchorHash,
    bytes32 anchorStateRoot, uint64 anchorTimestamp,
    bytes32 forceRoot, uint64 forceCutoff,
    uint64 startCursor,
    uint64 admissionVersion,
    bytes32 admissionRoot,
    uint64 episode, uint64 recoveryRevision, bytes32 recoveryId,
    bytes32 scheduleRootsCommitment, uint8 scheduleWindowCount,
    bytes32 sessionRefsCommitment, uint8 sessionCount,
    uint16 dataRecordCount, bytes32 executionOutputsCommitment,
    address proofBeneficiary)
```

`digestLimbs[0]` and `[1]` are respectively the high and low 128 bits of `statementHash`; neither is reduced modulo the proof field. The circuit recombines them and proves the exact preimage. Settlement independently checks the base canonical state, current/frozen versions and recovery round, recomputes the bounded schedule-root and sealed-session-reference commitments from calldata, and constructs the new core from the explicit (`tipHash`, `tipSlot`, `endStateRoot`, `endCursor`, `winningDataCommitment`) fields. It rejects zero/ill-typed fields, cursor regression, or disagreement with the last proved block, then stamps `canonicalizedAtBlock=block.number` outside the proof. `candidateId=statementHash`. `nextDueAt` is authenticated inside the frozen queue as the boundary leaf's deadline, or `UINT64_MAX` when `endCursor=forceCutoff`. Normal submission additionally verifies the current-root boundary proof described in §7; recovery uses the round's post-cutoff scalar. These values initialize the $O(1)$ head check. `proofBeneficiary` is a proof input, never builder-signed block data or coinbase. Copying proof bytes cannot redirect a later optional reward.

5.4 Preconfirmation semantics

A builder signature proves authorship and makes same-slot equivocation slashable. It does not prove that the body reached a prover or that the next builder saw it. A later scheduled builder may sign a profitable skip that the protocol cannot distinguish from non-receipt. Accordingly:

- before L1 candidate acceptance, a promise is *observed*;
- after acceptance and before close, it is *provisionally proved*;
- after normal close or recovery commit, it is *canonical*; and
- only the underlying L1 finality policy makes it *L1-final*.

Wallets and applications **MUST NOT** display the first two states as final. An escape commit explicitly revokes every omitted observed or provisionally proved promise and returns omitted

transactions to the mempool.

6 Blob data authentication

Data is posted into publisher-owned, bonded sessions. A session ID is

```
H("slot-chain-session-v1" || u256(chainId) || address20(contract) ||
  address20(owner) || u64(ownerNonce))
```

At most two sessions per owner and 1,024 globally may be live; each contains at most 2,100 records. Opening uses the first free ring cell after at most `MAX_GC_STEPS=8` deterministic expiry checks from a persistent cursor; callers may run bounded maintenance repeatedly. No call scans or clears all 1,024 cells. A full live ring rejects discretionary publication but cannot block the forced/escape lane. Dynamic rent, charged for the entire TTL and burned into local accounting, prices global occupancy. A Sybil willing to rent every cell can censor this optional DA market for one TTL; this is an explicit residual, not a protocol-liveness claim.

Only the owner may append, and a candidate may reference only a session made immutable by `sealSession`. Sealing fixes `(root, count, expiry)`; there is no unseal or append-after-seal operation. A candidate supplies at most 16 sorted unique `(sessionId, count, root)` references. At submission each must equal a live sealed session. Normal acceptance requires `expiry ≥ normalDeadline + REORG_MARGIN_SECONDS`; recovery requires `expiry ≥ block.timestamp + REORG_MARGIN_SECONDS`. Opening requires its expiry to cover proving, settlement and the reorg margin:

$$\begin{aligned} \text{expiry} &\geq \text{now} + P_PROVE_MAX + W_SETTLE_SECONDS \\ &\quad + REORG_MARGIN_SECONDS, \\ \text{expiry} &\leq \text{now} + DATA_TTL. \end{aligned}$$

For every blob attached to `postData(session, manifests[])`, the contract:

1. obtains `versionedHash=BLOBHASH(i)` from the same transaction;
2. computes legacy Keccak over this packed fixed-width sequence:

```
"slot-chain-data-fs-v2" || u256(chainId) || u256(protocolVersion) ||
sessionId || versionedHash || fullBodyRoot || u16(blockOrdinal) ||
u16(chunkIndex) || u16(chunkCount) || u32(chunkByteLength) ||
chunkRoot || address20(publisher) || u64(validUntil)
```

It interprets the digest big-endian and reduces it modulo the BLS scalar-field modulus to obtain z ;

3. calls the EIP-4844 point-evaluation precompile with exactly `versionedHash32 || z32 || y32 || commitment48 || proof48`; and
4. appends the exact leaf from §A to the session MMR.

The proof receives blob polynomials privately, recomputes $P(z) = y$, and decodes the canonical blob codec. A discretionary body byte string is `u32(txCount) || (u32(txLength) || signedRawTx)*`; its root is `H("slot-chain-body-v1" || u32(byteLength) || bodyBytes)`. Blob payload is `u32(chunkByteLength) || chunkBytes || zeroPadding`, split into 31-byte

chunks; each blob field element is `0x00||chunk31`. Exactly 4,096 elements are used, unused payload bytes are zero, and noncanonical encodings reject. A manifest entry binds (`blockOrdinal`, `chunkIndex`, `chunkCount`, `chunkByteLength`, `fullBodyRoot`, `chunkRoot`, `sessionId`, `recordIndex`). For each block entries are in strict chunk-index order, cover exactly $0 \dots \text{chunkCount}-1$, have $\text{chunkCount} \leq \text{MAX_CHUNKS_PER_BODY}=9$, and concatenate to at most $\text{MAX_BODY_BYTES}=1,142,748$. The circuit recomputes every chunk root, the full discretionary body root and then the full Ethereum transactions root. Manifests are ordered by block then chunk; duplicate, extra, overlapping or uncovered data is invalid. `dataManifestRoot` is the exact tree in Appendix A, not a free-form publisher value.

Data publication has no consensus-coupled reimbursement. A separate service may pay it, but empty payment state cannot revert canonical settlement.

7 Settlement and recovery state machine

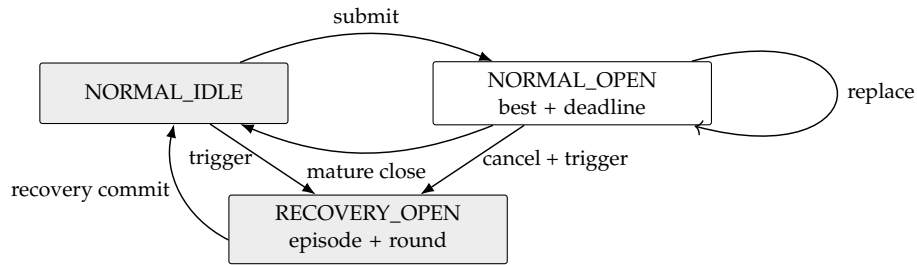


Figure 1: The complete mode machine. Payments are outside every arrow.

7.1 Normal candidates

The first valid tier-1 candidate extending the canonical tuple opens a normal window. It freezes `baseCanonical`, `deadline=block.timestamp+W_SETTLE_SECONDS`, and `minAdmissibleSlot=max(0,currentL2Slot-DELTA_TIP)`, and `current(admissionVersion,admissionRoot)`. It also freezes `requiredThrough=deadline+T_INCLUDE_MAX_SECONDS`. Every candidate first proves its frozen-prefix boundary in the circuit. It also supplies a canonical single-leaf Merkle proof at `endCursor` under the queue's current root/count, or proves `endCursor=currentCount`. Settlement verifies at most 32 sibling hashes and requires the resulting `liveNextDueAt` to exceed `requiredThrough`. This prevents an old but valid anchor from hiding a post-anchor append. Because due times are monotone, one current boundary leaf proves that every item due through the landing horizon is consumed. Later candidates use the same base, deadline, minimum, admission pair and horizon. Their total order is lexicographic:

(block count descending, tip slot descending, tip hash ascending).

Only a strictly better candidate replaces `normalBest`; the stored best contains its authenticated `endCursor`, `liveNextDueAt`. Provisional replacement changes no canonical state and pays nobody. At or after the deadline, `sync()` commits the stored best only if its boundary deadline is still greater than `block.timestamp`. The active queue router updates that one scalar when an append occurs exactly at the best's boundary. Thus a late close cancels rather than committing a block that omits an already-due head, even beyond the assumed inclusion bound. Because $\text{FORCE_DELAY} \geq \text{W_SETTLE_SECONDS} + \text{T_INCLUDE_MAX_SECONDS}$, an append after open cannot vi-

olate the frozen acceptance horizon. Settlement then recomputes recovery causes against the new core.

7.2 Activation

While normal, `sync()` opens one persistent recovery episode if either:

- `currentL2Slot > canonical.tipSlot + DELTA_FINAL_LAG`; or
- the forced queue head satisfies `enqueuedAt + FORCE_DELAY ≤ block.timestamp`.

A due forced head has precedence over an immature normal window, which is canceled. At a mature deadline, a best that consumes the required due prefix commits before causes are recomputed; an omitting best is canceled. No best is promoted across activation. The contract increments episode, records causes and the base-canonical hash, performs the SLA seat termination/burn at most once, then opens recovery round one. Termination and burn are fixed-size writes in Settlement-owned escrow accounting: no token transfer, registry callback or caller-controlled external call occurs on this path. A later sweep may dispose of recorded burns.

Every round freezes `roundStartSlot`, the preceding L1 block number/hash, its timestamp, current `forceRoot/forceCutoff`, current (`admissionVersion, admissionRoot`), and `escapeSlot = max(roundStartSlot + ESCAPE_OFFSET, canonical.tipSlot + 1)`. Its `expiresAt` is `GENESIS_TIMESTAMP + escapeSlot + DELTA_TIP`. The exact ID is:

```
recoveryId = H("slot-chain-recovery-v2", u256(chainId), address20(contract),
    u64(episode), u64(revision), bytes32(baseCanonicalHash),
    u64(roundStartSlot), u64(anchorNumber), bytes32(anchorHash),
    bytes32(forceRoot), u64(forceCutoff), u64(admissionVersion),
    bytes32(admissionRoot), u64(escapeSlot), u8(causes))
```

At `block.timestamp > expiresAt`, and never before, `sync()` replaces the expired target with revision $r + 1$ and fresh round fields, then returns SYNCED. It does not change episode/base/cause, re-burn, terminate, promote or pay. The old proof is already inadmissible, so permissionless refresh cannot grief a still-live proof. New queue appends enter the refreshed cutoff. An envelope admitted during recovery receives `dueAt = max(enqueuedAt + FORCE_DELAY, lastDueAt, currentRound.expiresAt + 1)`; therefore it cannot become due while a frozen round that omits it remains admissible.

7.3 Recovery acceptance

Recovery is first-valid, not heaviest. A tier-2 or tier-3 candidate MUST:

1. extend the exact current stored canonical state;
2. bind current episode, revision, `recoveryId`, frozen admission pair, anchor and queue target;
3. land only after `block.number - anchorNumber ≥ F_L1`, have first slot at least `roundStartSlot`, tip no older than `DELTA_TIP`, and no slot later than wall clock plus skew;
4. consume the exact maximum forced-message prefix from the current cursor, bounded by the current round's frozen cutoff; and

5. satisfy its tier-specific signature/data restrictions and validity proof.

The first valid L1 transaction atomically commits the tuple, records the current L1 block *number only*, clears the recovery episode and returns to normal. Later proofs are stale. Candidate rejection cannot undo activation because of the early-return wrapper in §3.

8 Forced queue and unsigned escape

ForcedEnvelope is a tagged union. Kind 0 contains (sender, nonce, chainId, rawTxHash, byteLength, gasLimit, maxFee, validUntil, refundAddress, deposit). Admission canonically decodes the EIP-2718 transaction, requires its recovered sender and all duplicated fields to match, and requires `msg.sender == sender`; relayed admission needs a separately versioned EIP-712 authorization. Kind 1 contains (msgHash, srcChainId, destChainId, srcOwner, destOwner, value, calldataHash, escrowId, byteLength, accountedGas, refundAddress, deposit). Only the configured BridgeInboxAdapter may enqueue it, it has no expiry field, and value is fully escrowed. Its only forced effect is an idempotent inbox/signal credit keyed by msgHash; arbitrary destination invocation and RETRIABLE processing remain later ordinary bridge operations. The two variants have separate byte-exact leaves.

Kind 0 requires a nonexpired validUntil and `accountedGas = max(gasLimit, intrinsicGas, MIN_FORCE_ACCOUNTED_GAS)`. Kind 1 uses the execution profile's fixed upper-bound InboxApply gas for its encoded credit. Both require $0 < \text{accountedGas} \leq 5,000,000$, $0 < \text{byteLength} \leq 131,072$, and a deposit covering execution, proof credit and permanent calldata/state costs. The adapter verifies kind-1 escrow before append; failure leaves neither queue leaf nor partial credit.

Only ActiveSettlementRouter may append. It calls the active Settlement's `sync()` under a non-reentrant guard and returns SYNCED without appending if any transition occurred. Otherwise it computes `dueAt = max(now + FORCE_DELAY, lastDueAt, recoveryExpiry + 1)` (the last term is zero outside recovery), appends, and calls the fixed-gas `onQueueAppend(index, dueAt)` hook. The hook updates the canonical-head, stored-best boundary, or current round's first-post-cutoff scalar only when its cursor equals index. A commit chooses the authenticated in-cutoff boundary or that recorded post-cutoff value, never overwriting the latter with `UINT64_MAX`. Thus `dueAt` is nondecreasing and force-due/coverage checks are $O(1)$, without a loop or per-item storage mapping in Settlement.

The queue is a depth-32 append-only Merkle vector with fixed capacity 2^{32} , stored root, count and 32-node append frontier. Admission rejects at capacity; an upgrade must occur before exhaustion, but already admitted items remain live. A canonical DFS range multiproof authenticates the consumed leaves and, when present, the first excluded boundary leaf under the frozen (`forceRoot, forceCutoff`). It supplies no fully covered subtree and no unused hash; at most 129 sibling hashes authenticate a 65-leaf block range. Skipping, reordering, changing the start index or claiming an early boundary changes the reconstructed root. Bytes are permanent enqueue calldata and their hash is in the leaf; blobs are forbidden on this availability path.

Each proved block consumes the unique maximum FIFO prefix satisfying all three bounds simultaneously:

$$\text{count} \leq 64, \quad \sum \text{byteLength} \leq 1,048,576, \quad \sum \text{accountedGas} \leq 20,000,000.$$

Every admitted item individually fits all three. The circuit exposes (start, end, count,

`totalBytes, totalAccountedGas, forceRoot, forceCutoff, nextDueAt`) and verifies the range multiproof. Maximality means either `end=forceCutoff`, or the authenticated next leaf would exceed at least one remaining per-block cap; an early stop with a fitting leaf rejects. Candidate totals are independently capped as stated in §5. Launch enforces $\text{ANCHOR_GAS_MAX} + \text{FORCE_GAS_BUDGET} + \text{SYSTEM_GAS_MARGIN} \leq \text{L2_BLOCK_GAS_LIMIT}$ and the analogous witness-byte bound.

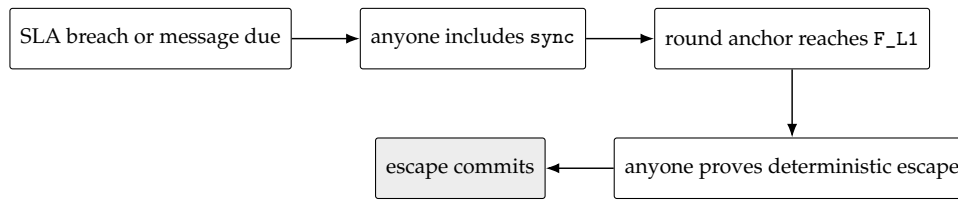
Forced envelopes are auxiliary protocol inputs, not blindly copied into the Ethereum transaction list. Each block's transaction order is: (1) the exact profile-defined AnchorV4 transaction, (2) one deterministic InboxApply system transaction, (3) upfront-valid kind-0 raw transactions in FIFO order, then (4) discretionary transactions. Sequential pre-state classification puts expired, wrong-nonce, underfunded or underpriced kind-0 items only in InboxApply's disposition vector; they have no Ethereum transaction, nonce or receipt. Upfront-valid kind 0 appears as an ordinary Ethereum transaction, whose success or revert has ordinary nonce/gas/receipt semantics. Kind 1 always produces its idempotent inbox credit inside InboxApply and never calls the eventual destination. The system calldata commits (`forceStart, forceEnd, (queueIndex, disposition, txIndexOrResultHash)*`); the circuit derives the transaction and receipt roots from this exact order. Unused deposit is later credited to `refundAddress`; failure there cannot change consumption.

For a normal signed block, `forceRoot/forceCutoff` are authenticated by its L1 anchor. For recovery they equal the current round's directly stored pair. The circuit consumes only up to that cutoff, so a later append cannot invalidate an already signed block or in-flight proof. All blocks in one candidate use the same cutoff. A due current head cannot be bypassed: `sync()` cancels an immature window, or closes a mature best only when its live boundary is not due, before opening recovery.

The 1,500-second force delay applies to the queue head, not to an arbitrary position behind existing prepaid work. An admitted sender can compute its worst-case number of escape blocks from the gas ahead at admission. As with L1 itself, the protocol does not promise backlog-independent latency against an adversary willing to buy all available capacity; it promises that builders cannot selectively veto the FIFO head.

An escape block is deterministic under a governance-frozen `executionProfileHash`. That profile commits the L2 fork rules; complete header-field derivation (gas limit, base fee, extra data, withdrawals/requests roots and randomness); the deployed AnchorV4 and InboxApply bytecode hashes; reserved senders, nonces and gas; AnchorV4 checkpoint calldata and `ancestorsHash` transition; and the transaction/receipt derivation above. For the initial profile the anchor is an EIP-1559 transaction from `0x0000777735367b36bC9B61C50022d9D0700dB4Ec`, gas limit 1,000,000, priority fee/value zero, max fee equal to the block base fee, empty access list, and the sender's sequential state nonce. It calls the frozen Anchor address with selector `0x523e6854` and ABI tuple (`uint48(anchorNumber), anchorHash, anchorStateRoot`) from the candidate's one batch anchor. Its deterministic protocol signature algorithm and the InboxApply transaction's corresponding fields are part of the profile vectors. Its timestamp is the round escape slot, coinbase is the protocol sink and there is no discretionary body. A repeated batch anchor still performs the exact per-block `ancestors` transition. Changing the execution profile requires a delayed protocol-version upgrade and new full-block golden vectors; runtime guessing is forbidden.

For a cartel with no usable builder block, the recovery sequence is:



With the initial bounds, activation inclusion (120 s), the frozen `ESCAPE_OFFSET=1,900s` (covering `T_DEPTH_MAX=900s`, proving at 900 s and 100 s margin), submission inclusion (120 s), and clock margin (24 s) total 2,164 seconds, below `DELTA_FINAL_LAG=3,600s`. Tip admissibility is separate: the escape slot is at the end of the offset, so only submission and skew—144 seconds—must fit `DELTA_TIP=1,200s` when proof generation meets its bound.

9 Economics, slashing and payments

Event	Objective evidence	Effect
Same-slot equivocation	Two valid distinct signed headers for one key/slot	Slash that schedule window's tranche once; tombstone key; emit reporter entitlement.
Aggregator SLA breach	Final lag exceeds threshold after mature close	Terminate and burn penalty bond once; open recovery; promote standby if any.
Force-only activation	Due queue head	No aggregator penalty; open recovery.
Skip or absence	None that distinguishes fault from non-receipt	Not slashable; promise may be revoked.
Invalid proof/data	Verifier failure	Reject; no canonical or payment effect.

Equivocation evidence is idempotent per (builder, window); separate windows slash separate tranches. `L_LEASE` is calibrated against at most 76 assigned slots but is not insurance for unbounded signed value. Tombstoning is atomic with the first valid slash; future snapshots exclude the key.

Canonical settlement performs no reward calculation, token balance write or external call. It always emits a candidate-committed event containing the ID, beneficiary and reward class. Best-effort protocol claims use a fixed `MAX_REWARD_RECEIPTS=256` ring, selecting `uint8(uint256(candidateId))`: if that cell is empty or its prior receipt is both claimed/expired and past the reorg margin, Settlement records the three fields, commit block and `claimed=false`; otherwise it only emits the event. Ring occupancy can therefore forfeit a reward but can never revert or delay canonical state. A separate distributor verifies this stored receipt, sets `claimed=true` before transfer, and pays at most the funded class cap before `REWARD_CLAIM_WINDOW=86,400s`. Logs alone are explicitly not on-chain-verifiable entitlements. Burns are never recycled into the same episode's bounty. Normal service/data compensation likewise cannot affect candidate validity or canonical commit.

10 Security and liveness analysis

Adversary/failure	Protocol result	Bound or residual
Invalid execution or forged state	Proof rejects.	Finalized-state safety reduces to proof-system and L1 safety.
Aggregator of-fine/byzantine	SLA activation terminates it; any prover uses tier 2 or 3.	At most final-lag threshold plus 2,164 s under stated inclusion/proof bounds.
No seat or no standby	State remains valid; escape has no seat dependency.	Same protocol bound; reward may be zero.
All builders collide/absent	Unsigned escape advances anchor and forced queue.	Meaningful discretionary throughput falls to forced envelopes; no builder censorship veto.
Builder equivocation	Competing signed branches; one may win normal order.	Direct per-window slash; pre-close promises remain revocable.
Builder skips/withholds body	Honest tail may be unprovable or omitted.	Not objectively slashable; eventual escape restores state progress but may revoke promises.
Data-session spam	Sessions are publisher-owned and bonded.	Attacker fills only its session; escape uses no blob session.
Transparent-mempool front-run	First valid recovery transaction wins.	Proof statement binds beneficiary, episode, revision and outputs; copying cannot redirect payment or state.
Payment-vault exhaustion	Claim is zero or partial.	Never blocks safety or canonical liveness; economic liveness is conditional.
L1 reorg within policy	Contract state, burns and claims roll back together.	Clients replay canonical logs and proofs against new version/hash.
L1 censorship/outage	Neither activation nor proof can land.	Outside model after T_INCLUDE_MAX_SECONDS; no L2 protocol can force its L1 contract to execute.

The escape lane establishes strong transaction-entry permissionlessness even when the capital-ranked builder set is captured. It does not make the normal builder market egalitarian: top-64 ranking and per-window capital remain a meaningful entry barrier. The product must distinguish these two claims.

11 Implementation blueprint

11.1 L1 modules and bounded storage

- **BuilderRegistry:** 64 active cells, 1,072 liability generations, 512-leaf tranche roots, tombstones, replacement counters and the versioned admissionRoot. Every transition is fixed-depth or constant-size.

- **ScheduleOracle:** sealed window root/seed in a ring covering `MAX_LIVE_WINDOWS=268`. `MAX_EARLY_SEAL_WINDOWS=8` covers the earliest snapshot geometry. In window units the capacity is at least:

$$\text{MAX_EARLY_SEAL_WINDOWS} + \text{MAX_CANDIDATE_WINDOWS} + \\ \text{ceil}((\text{W_SETTLE_SECONDS} + \text{DELTA_FINAL_LAG} + \text{DELTA_TIP} + \\ \text{REORG_MARGIN_SECONDS} + \text{EVIDENCE_DELAY_SECONDS}) / 384) + 2$$

An entry is overwritten only after it is `EXPIRED`; the same transaction checks the release predicate and no stored normal best/round reference.

- **DataSession:** a 1,024-cell global ring, at most two live cells per owner, each with owner/nonce ID, 2,100-leaf MMR frontier, sealed root/count and expiry; no on-chain leaf array. Expired eviction advances at most eight cells per call.
- **ForcedQueue:** depth-32 append-only Merkle root/count/frontier, last due time and deposits; payload bytes are permanent L1 calldata. Only the router appends.
- **Settlement:** `PREACTIVE` flag, canonical state, one normal best, recovery episode/round, seat pointer, queue-boundary scalars and a 256-cell best-effort receipt ring. Rewards and burn disposal are separate. No loop is proportional to chain age.

All setters enforce parameter inequalities atomically and are disabled after the launch freeze except through the existing delayed governance path. Every counter has an explicit width and checked increment; no sentinel shares a legal value.

11.2 Historical authentication

EIP-2935 authenticates normal candidate and schedule carrier headers. A recovery round captures its immediately preceding block hash with `blockhash(block.number-1)` and later compares candidates with that stored value and timestamp; it never queries an expired history entry or mutates the meaning of a past header. Its queue pair is round storage, not an MPT claim. EIP-4788 plus SSZ branches authenticates the schedule's parent beacon/execution payload; its state root plus MPT proofs authenticates the registry. Deployments lacking either primitive require an equivalently verified oracle and separate review.

11.3 Reorg rule

The canonical L1 log is the journal. A reorg truncates and replays, in order: canonical tuple, mode/version, normal best/deadline/frozen admission, episode/round, seat termination/burn, tombstone/tranche state, forced cursor, data-session root/count, and reward eligibility. Off-chain workers index by `(chainId, contract, blockHash, logIndex)` and discard orphaned jobs. Withdrawal/bridge execution waits for the configured L1 finality policy.

11.4 Genesis and migration

Deployment occurs at least `D_SNAP+F_FINAL+sealMargin` before activation, so first live sources are post-deployment and sealable. New Settlement begins `PREACTIVE`; registry, scheduling and data preparation may run, but every candidate rejects.

Forced ingress lives behind one `ActiveSettlementRouter`; the queue itself is shared across the cutover. Existing Taiko bridge messages remain on the legacy `SignalService/Bridge` retry path and are not silently converted into kind 1. Only new, explicitly escrowed inbox-credit

messages use `BridgeInboxAdapter`; its finalized message manifest binds the existing message hash and ownership/value/calldata fields, while destination invocation remains a later bridge operation.

Migration has two phases. First, anyone calls the ordinary entry point until `sync()` returns unchanged and the old contract is quiescent; this may take more than one transaction and never bypasses the early-return rule. Second, the governance executor calls `old.freezeAndExport(new)`. It asserts that no objective close/activation/round-roll is pending, cancels only a still-immature provisional best, disables old candidate ingress, exports the exact canonical state/cursor, makes the new contract verify the shared queue pair and an indexed boundary proof for `headDueAt` (or empty-at-count), atomically switches router authority, and changes new `PREACTIVE` to `NORMAL`. The operation contains only bounded non-reverting internal writes after its assertions; any revert rolls all contracts back. Old bridge signals keep their original claim semantics, so the migration makes no unenforceable claim to enumerate all historical pending messages.

12 Parameters and enforced geometry

Parameter	Initial value	Enforced relation
L2 slot / schedule window	1 s / 384 slots	Aligned by genesis timestamp.
N_MAX, Q_MAX	64 / 76	Six addresses fill 384 tickets.
BOND_MAX	$2^{192} - 1$	$\text{bond} * (\text{Q_MAX} + 1) < 2^{256}$.
Entry/exit delay; generation moves	8 / 268 windows; 4 per window	Fixed liability capacity is 4×268 .
D_SNAP, F_FINAL	8 / 2 L1 epochs	Strictly exceeds scan + finality + seal margin + lookahead.
H_LOOK	768 L2 slots	Guaranteed by snapshot geometry with one-window slack.
G_MAX	64 L2 slots	Tier-1 parent gap only.
W_SETTLE_SECONDS	1,200 s	Timestamp deadline, never L1 block count.
DELTA_TIP	1,200 s	At least submit inclusion + skew = 144 s after the frozen escape slot.
DELTA_FINAL_LAG	3,600 s	At least activation + escape offset + submit + skew = 2,164 s.
F_L1; T_DEPTH_MAX	64 L1 blocks; 900 s	Stored recovery-anchor depth and assumed maximum time to reach it.
ESCAPE_OFFSET	1,900 s	At least depth-time bound + proof bound + margin.
FORCE_DELAY	1,500 s	At least settlement window plus T_INCLUDE_MAX_SECONDS=120.
Forced prefix	64 items; 1,048,576 B; 20m gas	Each item at most 131,072 B and 5m accounted gas.

Candidate caps	4,096 blocks; 12 windows; 1 anchor; 16 sessions; 2,100 records	Also 256 forced items, 4 MiB, 80m accounted gas.
Queue / range proof	depth 32 / at most 129 hashes	Capacity 2^{32} ; canonical contiguous range proof.
Body chunks	9 / 1,142,748 B	Per-body chunks / maximum discretionary bytes.
DATA_TTL	86,400 s	Greater than candidate, settlement, proof and reorg path.
REORG_MARGIN, EVIDENCE_DELAY	1,800 / 86,400 s	Data resubmission and finite liability retention.
Live windows / sessions	268 / 1,024	Fixed rings with safe-eviction predicates and bounded GC.
EIP history	8,191 entries	Greater than every normal-anchor and schedule path.

Launch tooling evaluates every inequality in both seconds and its native clock unit. Governance cannot set a value that violates a relation.

13 Production gates and verdict

The consensus design is fixed enough to implement. Production deployment is blocked until every gate below produces a versioned artifact and passes:

1. **Proof performance.** Independent tier-1, tier-2 and worst-case tier-3 proofs, including 4,096 blocks and 2,100 data records, finish within 900 seconds on the published minimum hardware. Peak RAM, recursion setup and failure recovery are measured; a miss requires changing parameters and another design review.
2. **Contract gas.** `sealWindow`, `postData`, normal submit, sync, recovery submit, slash and garbage collection fit current L1 gas limits with 30% margin. Fuzzing includes empty registry, zero seat, full data session, full queue prefix and maximum history age.
3. **Cryptographic conformance.** Solidity, Go, Rust and circuit code match the Keccak/EIP-712 golden vectors, SSZ generalized indices, MPT proofs, KZG Fiat–Shamir transcript and exact body manifest. Two independent implementations generate every vector.
4. **State-machine verification.** Stateful fuzzing models EVM rollback, same-block ordering, reorg replay, stale versions, activation/close coincidence, message consume-and-discard and storage reclamation. The executable models in Appendix D remain green.
5. **Economics.** Auction bond, per-window tranche, forced-envelope deposit, storage bond and proof credits are calibrated under blob/L1 fee spikes. The published liveness claim explicitly becomes conditional above fee caps.
6. **Operations.** At least two independent prover stacks, public data session mirrors, schedule sealers and recovery bots complete a shadow-fork outage exercise. The exercise removes every builder and seat, forces a user transaction, reorgs the first recovery attempt, and still reaches finality.
7. **External review.** Separate proof-system, Solidity, bridge and economic audits close all crit-

ical/high findings; the migration rehearsal and emergency governance path are in scope.

Verdict. *The earlier fallback architecture was not implementation-ready: it depended on a scheduled builder, could revert activation inside a rejected submission, made payment a commit precondition, and could not bind whole-window data with a six-blob cap. This revision removes those dependencies and fixes a single consensus path. It is implementation-ready as a specification. It is not production-ready as a deployed system until the seven empirical gates pass; claiming otherwise from a document and Python assertions alone would be unsound.*

A Normative commitment encodings

All integers in packed commitments are unsigned big-endian at the named width; addresses are 20 bytes, hashes are 32 bytes, and ASCII domains have no length prefix or trailing NUL. `H` is Ethereum legacy Keccak-256. No `abi.encodePacked` call may contain a dynamic field. Raw transaction bytes are represented by `H(rawTx)` plus an explicit `u32(byteLength)`.

The canonical core/base, candidate, schedule list, sealed-session list and execution outputs use these exact encodings:

```
coreHash = H("slot-chain-core-v2" || tipHash || u64(tipSlot) || stateRoot ||
  u64(messageCursor) || winningDataCommitment)
baseCanonicalHash = H("slot-chain-canonical-v2" || coreHash ||
  u64(canonicalizedAtBlock))
H("slot-chain-candidate-v2" || baseCanonicalHash || u16(n) ||
  (u64(slot) || blockHash || bodyRoot || u64(messageEnd))* )
H("slot-chain-schedule-list-v1" || u8(count) ||
  (u64(window) || entryRoot || seed)* )
H("slot-chain-session-list-v1" || u8(count) ||
  (sessionId || u16(recordCount) || root)* )
H("slot-chain-outputs-v1" || stateRoot || transactionsRoot || receiptsRoot ||
  logsBloomHash || withdrawalsRoot)
```

Here $n \leq 4096$; schedule windows are ascending and sessions are ascending by ID. These hashes are respectively the statement's `baseCanonicalHash`, `candidateCommitment`, `scheduleRootsCommitment`, `sessionRefsCommitment` and `executionOutputsCommitment`; unsorted or duplicate lists reject.

The 64-cell registry leaf at index i is

```
H("slot-chain-registry-leaf-v1" || u8(i) || u8(present) ||
  address20 || u192(bond) || u64(registrationIndex) ||
  u64(effectiveL2Slot) || bytes32(trancheRoot) || u64(tombstonedAtL2Slot))
```

An empty cell has `present=0` and every following byte zero. A present cell has `present=1`; `UINT64_MAX` means not tombstoned. At tree height $h = 0, \dots, 5$, the parent is:

```
H("slot-chain-registry-node-v1" || u8(h) || left || right)
```

There is no padding because there are exactly 64 leaves.

Admission position $i < 1,136$ is:

```
H("slot-chain-admission-leaf-v1" || u16(i) || u8(present) ||
  u8(location) || address20 || u192(bond) || u64(registrationIndex) ||
  u64(effectiveL2Slot) || trancheRoot || u64(tombstonedAtL2Slot))
```

Location is 1 for active and 2 for liability; an empty leaf has `present` and all following fields zero. Positions 1,136...2,047 and unused positions are canonical empty leaves. Its eleven levels use `H("slot-chain-admission-node-v1" || u8(height) || left || right)`. `admissionVersion` increments before publishing the corresponding root; the pair is stored together and never reconstructed from a newer root.

The omission-free ranked-entry commitment for window W has exactly 64 leaves:

```
H("slot-chain-entry-leaf-v1" || u8(rank) || u8(present) || address20 ||
  u192(bond) || u64(registrationIndex) || u64(effectiveL2Slot) ||
  u64(tombstonedAtL2Slot) || bytes32(trancheLeafHash))
```

followed by six levels of $H(\text{"slot-chain-entry-node-v1"} || u8(\text{height}) || \text{left} || \text{right})$. Empty ranks have all fields zero. The schedule circuit derives all 384 tickets from this root and seed; a schedule list cannot substitute a different set.

Each registry cell's tranche ring is a 512-leaf tree. Leaf index j is:

```
H("slot-chain-tranche-leaf-v1" || u16(j) || u64(window) || u8(state) ||
  u192(amount) || u64(liableUntil))
```

An empty leaf has `window=UINT64_MAX`, state zero and remaining fields zero. Its nine node levels use $H(\text{"slot-chain-tranche-node-v1"} || u8(\text{height}) || \text{left} || \text{right})$. The snapshot supplies one inclusion proof for index $W \bmod 512$ for every active cell, including canonical empty and FREE leaves.

Kind-0 forced leaf i is legacy Keccak over this exact packed sequence:

```
"slot-chain-force-user-v2" || u32(i) ||
address20(sender) || u64(nonce) || u256(chainId) || bytes32(rawTxHash) ||
u32(byteLength) || u64(gasLimit) || u64(accountedGas) ||
u256(maxFee) || u64(validUntil) || address20(refundAddress) ||
u64(enqueuedAt) || u64(dueAt) || u256(deposit)
```

A kind-1 leaf is:

```
"slot-chain-force-bridge-v2" || u32(i) || msgHash ||
u256(srcChainId) || u256(destChainId) || address20(srcOwner) ||
address20(destOwner) || u256(value) || calldataHash || escrowId ||
u32(byteLength) || u64(accountedGas) || address20(refundAddress) ||
u64(enqueuedAt) || u64(dueAt) || u256(deposit)
```

Its application has no expiry; no `validUntil` field exists.

The depth-32 vector empty leaf is $H(\text{"slot-chain-force-empty-v2"})$. A parent at height $h = 0, \dots, 31$ is $H(\text{"slot-chain-force-node-v2"} || u8(h) || \text{left} || \text{right})$. The queue commitment is $H(\text{"slot-chain-force-root-v2"} || u64(\text{count}) || \text{treeRoot})$; leaves at indices $\text{count}..2^{32}-1$ are empty. The canonical range proof recursively visits the smallest tree covering $[\text{start}, \text{end}]$ (including a boundary leaf when $\text{end} < \text{count}$); it emits in DFS order exactly one hash for each maximal outside subtree. The verifier rejects extra/missing nodes, a nonempty padding leaf, or a reconstructed root/count mismatch. Append at old index i starts with the new leaf at height zero, repeatedly hashes `frontier[h]` on the left while bit h of i is one, stores the carry in the first zero-bit frontier position, increments count, and reconstructs all higher right subtrees from `FORCE_EMPTY[h]`. This is the same root as the full vector and uses exactly 32 frontier words.

A data-chunk leaf is:

```
H("slot-chain-data-leaf-v1" || bytes32(sessionId) || u16(index) ||
  bytes32(versionedHash) || bytes32(fullBodyRoot) || u16(blockOrdinal) ||
```

```

u16(chunkIndex) || u16(chunkCount) || u32(chunkByteLength) ||
bytes32(chunkRoot) || address20(publisher) ||
u64(validUntil) || u256(z) || u256(y))

```

MMR append merges equal-height peaks as $H(\text{"slot-chain-data-node-v1"} || u8(\text{height}) || \text{left} || \text{right})$. Peaks are stored left-to-right in strictly decreasing height. The root bags them rightmost-to-leftmost as $H(\text{"slot-chain-data-bag-v1"} || u16(\text{count}) || u8(\text{peakCount}) || (u8(\text{height}) || \text{peak})^*)$. An inclusion proof provides the leaf index, merge siblings bottom-up with a left/right bit at each height, then every other peak in this bag order; duplicate heights, a count inconsistent with peak heights, or unused proof elements reject. The empty MMR is the bag hash with zero count and zero peaks.

A manifest leaf is:

```

H("slot-chain-manifest-leaf-v1" || u16(position) ||
  u16(blockOrdinal) || sessionId || u16(recordIndex) ||
  u16(chunkIndex) || u16(chunkCount) || u32(chunkByteLength) ||
  fullBodyRoot || chunkRoot)

```

Its next-power-of-two tree uses $H(\text{"slot-chain-manifest-node-v1"} || u8(\text{height}) || \text{left} || \text{right})$ and the root is $H(\text{"slot-chain-manifest-root-v1"} || u16(\text{count}) || \text{treeRoot})$; padding uses the domain-separated empty leaf. The InboxApply disposition commitment is:

```

H("slot-chain-dispositions-v1" || u64(start) || u64(end) ||
  u16(count) ||
  (u64(queueIndex) || u8(disposition) || u32(txIndex) || resultHash)^*)

```

Here `UINT32_MAX` means no Ethereum transaction.

The recovery ID is the encoding in §7. Schedule leaf/root, seed and tie hashes are as in §4; body and blob encodings are as in §6. `commitment-model.py` is the normative executable fixture. Its initial vectors are:

TYPEHASH	0a0bfc3b8ba52a166f662055d09891465d916aa486808e78105890e98d65777f
registryRoot	0bf297d7b9b6a5529a319a06cb08484923a89bab15d51f8baeaf5c30bebdf3fd
admissionRoot	783548941546bbc76487459185c787e4ffd7677b4487692dd6b1308dcc2e13f6
entryRoot	2eec2b75d264b328ec558d880b822ece2f85661516b1d85056a6f20ecd0dc7f1
forcedRoot	ab03f105ee7d619fb2c81d31b38760720dff2cb35471bc28fdc063c31f71bd67
manifestRoot	bb06a375def8deb09a550f685a65096eaf14cc70816d0592c978c51f96dcb1be
statementHash	98a0e1a892e2fd43e569f64dce3478ba19636fd062aa6ca517c49e6485fad22a
recoveryId	9c2f404af0886c6f7abb51f1bbe578747b7122dbe792ae9e5f5d4e7b74c8f550
bodyRoot	0f4e161a46c8b18c2a86f23a0a4e7169a838a12af8b389f65e97b547a99707e9

Release CI must reproduce every vector in Solidity, Go, Rust and the circuit.

B Normative transition ordering

Every external candidate entry point implements the following structure:

```
function submit(input) returns (Status) {
```

```

    if (mode == PREACTIVE) return REJECTED_PREACTIVE;
    if (sync()) return SYNCED;      // successful transaction, never revert
    if (mode == NORMAL) return submitNormal(input);
    return submitRecovery(input);
}

function sync() returns (bool changed) {
    if (mode == RECOVERY && now > round.expiresAt) {
        rollRound();                // same episode/base/burn; new target
        return true;
    }
    if (mode == NORMAL && forcedHeadDue && normal window immature) {
        cancel normal window;
    } else if (normal deadline matured) {
        if (best exists && best.nextDueAt > now) commit(normalBest);
        else cancel normal window;
    }
    if (mode == NORMAL && (finalLagBreach || forcedHeadDue)) {
        openRecoveryEpisodeAndRound();
        if (finalLagBreach) terminate incumbent once;
        return true;
    }
    return whetherCloseOrCancelChangedState;
}

```

No transfer, reward calculation, unbounded loop or caller-controlled external call occurs in canonical settlement. Optional reward materialization is a later transaction in a separate module.

C Rejected alternatives

1. **Heaviest fallback.** Rejected because an observer cannot prove it has seen the globally heaviest off-chain tail; transparent races can prevent a stable target. Recovery is episode-bound first-valid.
2. **Scheduled-builder-only fallback.** Rejected because the same cartel that stalls normal production also controls recovery. Tier 3 is deterministic and unsigned.
3. **Promote the pre-activation normal best.** Rejected because it was accepted under different cutoff/version semantics. Only a mature best that covers the required forced prefix closes; immature state is canceled.
4. **Atomic reward-funded commit.** Rejected because an empty seat or vault becomes a consensus deadlock. Reward materialization is a separate non-authoritative transaction.
5. **One same-transaction blob set.** Rejected because Ethereum exposes at most a handful of blobs per transaction while a candidate spans thousands of blocks. Publisher-owned authenticated sessions separate posting from proof.
6. **One KZG opening at a prover-chosen point.** Rejected because it does not soundly tie declared bodies to the blob. The challenge commits to both blob hash and body root, and the circuit recomputes the evaluation and body root.

7. **Arithmetic snapshot slot without EIP-4788 parent semantics.** Rejected. A bounded forward carrier search authenticates the parent source or fails closed.
8. **Bond as insurance.** Rejected without an on-chain reservation ledger. The tranche is deterrence only.

D Executable consensus models

`lookahead-model.py` reports 34 assertions over legacy Keccak, EIP-4788 carrier/parent selection, immutable seal deadlines, post-seal tombstones, eligibility-before-ranking, capped D'Hondt and feasible placement, including absolute clock conversion, execution-block finality depth, fixed seed and complete-schedule vectors.

`settlement-window-model.py` reports 50 assertions over PREACTIVE/migration, early-return semantics, force-due precedence, renewable recovery, no-seat escape, bounded Merkle-prefix predicates, immutable anchor chronology, $O(1)$ due boundaries, late close, sealed data sessions, frozen admission pairs, bounded replacement/liability generations, tranche release units, replay and timing geometry. Cryptographic booleans in that model are not cryptographic evidence.

`commitment-model.py` reports 35 vectors/properties for exact EIP-712 domain/struct/digest, canonical and statement hashes, registry/admission/entry/ tranche commitments, forced vector/range proofs, session MMR, manifest, dispositions, recovery ID, Fiat-Shamir input, body chunks and blob codec. The valid KZG-opening, signature-recovery, independent-implementation and proof gates remain mandatory.

E Security invariants checklist

1. Canonical state changes only after one valid proof and only from its stored base tuple.
2. A sync transition cannot be rolled back by validation of the same call's candidate.
3. No payment, seat or standby is required for canonical commit.
4. Recovery candidates bind one live episode/revision, the current base tuple, one stored prior-block anchor at required depth and one frozen queue target. Expired rounds are permissionlessly renewable without another burn.
5. A due forced head advances in the next canonical block; appends omitted by a frozen recovery round cannot become due until that round expires. Prefix count, bytes and accounted gas are independently bounded.
6. Tier 3 has no builder signature, discretionary transaction, arbitrary coinbase or blob body.
7. Every tier-1/2 block has a valid scheduled signature under the frozen admission version/root pair, strict slot progress and the candidate's one authenticated anchor no later than its L2 time.
8. Every discretionary body byte is covered exactly once by a unique, unexpired authenticated data record.
9. Schedule inputs share one authenticated execution payload and use exact Keccak encodings.
10. Tombstoned keys cannot re-enter; each window has an independent slashable tranche and finite release state.

11. Candidate, window, data-session, Merkle range proof, queue count/bytes/gas, history and storage-retention work is bounded by consensus constants and safe eviction.
12. L1 reorg replay removes canonical state, penalties and credits from orphaned transactions together.
13. After ordinary quiescence, migration freezes old ingress, imports canonical/bridge cursors into PREACTIVE settlement and switches the one active queue router atomically.